Masar: A Lightweight Sequence-Model Approach to Real-Time Pilgrim Safety Monitoring in Hajj Campaigns

Faisal Almars
Dept. of CS and AI
University of Jeddah
Jeddah, Saudi Arabia
2240087

Safwan Alimam
Dept. of CS and AI
University of Jeddah
Jeddah, Saudi Arabia
2242740

Abdulaziz Etaiwi
Dept. of CS and AI
University of Jeddah
Jeddah, Saudi Arabia
2140040

Omar AlSharqawi
Dept. of CS and AI
University of Jeddah
Jeddah, Saudi Arabia
2240754

Abdulrahman AlSumairi
Dept. of CS and AI
University of Jeddah
Jeddah, Saudi Arabia
2243821

***Abstract*—The annual Hajj pilgrimage is one of the largest recurring mass gatherings on Earth, drawing more than two million pilgrims into tightly constrained spaces and time windows. Campaign supervisors still rely on manual methods such as roll calls, paper checklists, and phone communication to track pilgrims, which are time-consuming and error-prone in dense, high-mobility environments. Existing smart solutions either depend on heavy video-based crowd analytics or on expensive wearable infrastructure that cannot scale down to the small and mid-sized campaign level. This paper presents *Masar*, a lightweight AI-powered tracking and anomaly-detection system that uses smartphone GPS together with the supervisor's live position as a dynamic geofence. We frame supervision as a streaming sequence-classification problem and compare three architectures whose inductive bias matches time-ordered GPS: a vanilla Recurrent Neural Network (RNN), a Long Short-Term Memory (LSTM) network, and a compact Transformer encoder. The three models share an identical training pipeline, feature contract, and route-disjoint evaluation protocol so that only the temporal mixer changes. Trained on a 90,000-row synthetic Hajj-movement dataset spanning 75 routes and evaluated on completely unseen trajectories, the three sequence models cluster within ± 0.003 F1 of each other (LSTM 0.934, Transformer 0.932, RNN 0.932) and all outperform a distance-threshold reference (0.928). Beyond raw accuracy, we describe the end-to-end production architecture — a React frontend, a FastAPI + SQLite backend with WebSocket fan-out, role-based authentication, and a dynamic supervisor-anchored geofence — and discuss the deployment, privacy, and operational implications of running such a system on commodity smartphones. The full implementation is open-source and reproducible.**

***Index Terms*—Hajj, pilgrim tracking, anomaly detection, sequence models, RNN, LSTM, Transformer, GPS, real-time monitoring, dynamic geofencing, smart pilgrimage, Vision 2030.**

I. Introduction

The Hajj pilgrimage is among the world's largest annual mass gatherings: the Day of Arafat alone draws over one million pilgrims to a single site, and the Mina valley hosts comparable densities during the stoning ritual. The combination of crowd density, restricted ritual time windows, language diversity among foreign pilgrim groups, and complex spatial logistics creates persistent safety and coordination challenges that have repeatedly turned into casualty events in modern history.

Despite substantial digital investment by the Saudi government, the operational backbone of most pilgrim campaigns remains manual. Campaign supervisors — who shepherd groups of 30–200 pilgrims through the rites — still rely on roll calls, paper checklists, WhatsApp group chats, and informal counts to monitor presence and movement. These methods are time-consuming, error-prone, and ineffective in crowded or fast-moving environments. Existing smart solutions focus largely on either (i) large-scale video-based crowd analytics requiring fixed infrastructure deployed by central authorities, or (ii) specialised wearables that are too expensive to roll out across the long tail of small and mid-sized campaign operators. Conventional commercial real-time location systems require continuous internet connectivity and consume high device power, making them impractical in the field where cellular coverage is congested and battery is scarce.

A. Problem Statement

The supervisor's problem is fundamentally one of *streaming awareness*: given a continuous flow of GPS readings from each pilgrim in the group, decide *at each moment in time* whether that pilgrim's recent trajectory is consistent with the group's movement or whether the pilgrim is at risk of separating. The answer must arrive within seconds, must remain stable under noisy GPS, and must be conveyed in a form that a supervisor coordinating a moving group can act on. No existing publicly-described system targets this specific niche — the small-campaign supervisor with a smartphone, a low-cost GPS bracelet, or both.

B. Contributions

This paper presents Masar, an end-to-end system that addresses the gap above. Our contributions are:

1) A practical end-to-end architecture combining smartphone GPS streaming, real-time feature engineering, a sequence classifier, and a WebSocket-driven supervisor

dashboard, designed to run on commodity hardware and tolerate intermittent connectivity.
2) A *dynamic geofence* mechanism anchored on the supervisor's own live GPS position rather than a fixed coordinate, together with a staleness-aware fallback that refuses to emit confident labels when the anchor is missing.
3) A direct, like-for-like comparison of three sequence architectures — vanilla RNN, LSTM, and a compact Transformer encoder — under identical training, feature, and split conditions, alongside an explicit argument for why classical tabular classifiers are inappropriate baselines for the streaming Hajj-supervision problem.
4) A reproducible evaluation on a route-disjoint test split of a 90,000-row synthetic pilgrim-movement dataset, with per-model confusion matrices, training curves, and error analysis.
5) A discussion of deployment, privacy, and operational considerations for systems of this class, including a threat model for the supervisor-pilgrim trust relationship.

The remainder of the paper is organised as follows. Section II surveys related work in Hajj-specific tracking, GPS-based anomaly detection, and crowd analytics. Section III describes the system architecture. Section IV details the dataset, features, and online inference protocol. Section V introduces the three sequence models and the rationale for restricting the comparison to them. Section VI reports results. Section VII discusses deployment considerations and limitations. Section VIII outlines future work and Section IX concludes.

II. Related Work

We organise the related literature into four threads: GPS- and IoT-based pilgrim tracking, AI-based crowd analytics, wearable physiological monitoring, and macro-level studies of AI in mega-events.

A. GPS- and IoT-based Pilgrim Tracking

Qadr and Wibowo [4] couple GPS-enabled IoT devices with a cloud dashboard for real-time pilgrim tracking, achieving roughly 2 m positional accuracy. Their pipeline assumes always-on connectivity to a cloud platform and relies on manual administrative oversight to handle exceptions; there is no automated mechanism to predict separation before it happens. Parveen and Aldhlan [5] present a low-cost prototype combining GPS, GSM modems, and an Arduino microcontroller to locate *already-missing* pilgrims via SMS. The system is purely reactive: it triggers only after a pilgrim has been manually reported lost and depends entirely on GSM coverage, which becomes unreliable inside the Holy Mosques during peak crowd density.

Both of these approaches share two limitations relative to Masar. First, they treat each GPS point in isolation rather than as part of a trajectory — a single point cannot distinguish a brief pause from genuine separation. Second, they require either commercial cloud infrastructure or fixed GSM/SMS pipelines, which adds operational cost that small campaign operators rarely absorb.

B. AI-based Crowd Analytics

Alzahrani and Algethami [1] propose a hybrid Isolation Forest + LSTM model for abnormal-movement detection from video data, reporting 91% accuracy. The pipeline depends on multi-camera deployments and substantial GPU compute, which restricts its applicability to centrally-operated venues. Shah [2] classifies crowd density in Hajj video frames with 87% accuracy using a lightweight CNN, again tied to video infrastructure controlled by the General Authority for Statistics rather than by individual campaign supervisors. A broader survey [10] catalogs AI applications across Hajj and Umrah rituals at a high level, identifying gaps in real-time implementation but without offering reproducible benchmarks. None of these address the specific problem of *per-pilgrim* risk monitoring inside small groups.

C. Wearable and Physiological Monitoring

Elgamal [3] demonstrates a wearable-based mobile monitoring application reaching 97.7% accuracy on fatigue prediction during Hajj. Al-Shaery et al. [8] integrate wearable physiological sensors, mobile technology, and an AI backend for real-time pilgrim health monitoring. Alharthi et al. [9] extend this line of work with a detailed analysis of pilgrim stress and fatigue patterns using wearable remote sensing. All three approaches deliver strong predictive signal but require specialised hardware that has to be acquired, charged, fitted, returned, and sanitised across the Hajj season — a logistic burden that excludes them from the small-campaign segment.

Chipless RFID tags [6] reduce per-device hardware cost dramatically but require fixed reader infrastructure deployed along pilgrim paths, which makes them an asset of the host authority rather than the campaign. The Personal Digital Mutawwif app [7] provides culturally-aligned navigation guidance using location services but has no predictive safety layer.

D. Macro-level Studies and Policy Context

Khan and Shambour [11] and Abalkhail and Al-Amri [12] review AI deployments in mega-events and Saudi Arabia's broader management of the Hajj season through AI and sustainability initiatives. Altoaimi [13] discusses geospatial data management for historic pilgrimage routes. These works frame the policy and infrastructure landscape and explain the strategic alignment with Saudi Vision 2030 [14], but they do not provide campaign-scale technical solutions.

E. Gap Analysis

Across this literature, four gaps persist for the campaign-level supervisor:

- **Connectivity dependence.** Most existing systems assume always-on cloud connectivity, which is unreliable in the dense areas of Mina and Arafat.
- **Reactive rather than predictive.** GPS+GSM trackers wait until a pilgrim is reported lost; video crowd analytics monitor aggregates, not individuals.

- **Cost asymmetry.** Wearables and chipless RFID infrastructure place capital expenditure on the smallest economic actor in the system: the campaign operator.
- **No published benchmarks for the per-pilgrim task.** Existing AI work targets either fatigue or crowd density; the question "is *this specific pilgrim* drifting away from the group?" is largely unaddressed in the literature.

Masar targets exactly this gap with a smartphone-only, AI-driven, partially offline-tolerant design with reproducible benchmarks.

III. System Architecture

Masar is organised in three layers: a React + Leaflet frontend serving three role-specific user interfaces, a FastAPI + SQLite backend with WebSocket fan-out, and a PyTorch-based AI inference service. The three layers communicate over a single HTTP/WS surface so that pilgrim devices, supervisor dashboards, and the inference loop can be reasoned about as independent processes.

A. Frontend

Three role-specific UIs are served from the same React 18 + Vite bundle.

- **Pilgrim client.** A lightweight mobile web page engineered for low battery usage. It streams GPS readings to the backend and surfaces only essential status indicators (*Normal* / *Abnormal*) along with a one-tap SOS button. The page is intentionally a Progressive Web App rather than a native app: this avoids the appstore-review friction and storage overhead that have historically deterred pilgrims from installing campaign-specific software.
- **Supervisor dashboard.** Built with `react-leaflet`, it renders live pilgrim positions, AI labels, and per-pilgrim distances around a marker centred on the supervisor's own GPS fix. Pilgrims whose label flips to *Abnormal* are highlighted in red and float to the top of the list; pilgrims who have requested SOS show a separate badge.
- **Admin console.** Manages campaigns, users, and the assignment matrix of pilgrims to supervisors. Account provisioning is bcrypt-based password authentication; the admin emits user IDs that supervisors and pilgrims use to log in.

Role-gated routing is enforced by a custom `useAuthGuard` hook that reads the local-storage authentication token and redirects on a role mismatch.

B. Backend

The backend is a FastAPI service backed by SQLite (via SQLAlchemy async sessions) and an in-process WebSocket connection manager. Five endpoint families are exposed:

- `/api/auth` — password-based login.
- `/api/admin` — campaign / user / assignment CRUD.
- `/api/supervisor` — live position sharing and per-supervisor queries.
- `/api/pilgrim` — status reads and SOS submission.
- `/api/location` — the main GPS-ingest path, described in detail below.

The WebSocket manager routes a single `pilgrim_update` payload to (i) any connected admin dashboard, (ii) the assigned supervisor's connection, and (iii) the pilgrim's own connection in parallel, so all three views update within the same cycle. An in-memory cooldown in the alert service prevents duplicate `AlertEvent` rows when a pilgrim flickers between Normal and Abnormal within the same 30-second window.

C. Dynamic Geofence via Supervisor GPS Sharing

Rather than fixing the geofence around a pre-configured anchor, Masar anchors it on the supervisor's live phone GPS. The supervisor publishes location updates via `POST /api/supervisor/{id}/location`; the backend keeps an in-memory `_supervisor_positions` dictionary keyed by user id. When a pilgrim's reading arrives without explicit supervisor coordinates, the backend backfills from this store — but only if the stored fix is younger than 30 s; stale fixes are flagged `supervisor_gps_stale` rather than silently used, because a hallucinated supervisor position would render every downstream distance computation meaningless. If no supervisor position is available at all, the system explicitly broadcasts `supervisor_missing` and refuses to emit a confident Normal label, deferring instead to a "waiting for supervisor" state on the dashboard.

This design choice has a security implication: a malicious or compromised pilgrim device cannot forge a Normal label by providing fake supervisor coordinates, because the supervisor coordinates are taken from the supervisor's authenticated session rather than from the pilgrim's payload.

D. AI Inference Service

The inference service exposes a single `predictor.update(pilgrim_id, reading)` call. Internally it maintains a per-pilgrim `deque` of feature vectors (capacity 10) and an LSTM checkpoint that is loaded once at startup. Each new reading triggers the sequence in Algorithm 1. Predictions before the window fills are returned as `window_full=False` with `label=None`, which the dashboard renders as a "warming up" state.

The LSTM is the deployed default; the RNN and Transformer trained alongside it (Section V) share the same I/O contract and are swap-in replacements.

E. Persistence and Audit Trail

Two relational tables back the inference path: `location_events`, which stores every GPS reading along with the resolved supervisor coordinates and emitted label, and `alert_events`, which stores Abnormal-state notifications with their acknowledgment status. The schema is intentionally append-only on the hot path so that an after-action review can reconstruct exactly which evidence produced which alert. SQLite was chosen over a hosted database for two reasons: it removes a deployment

Algorithm 1 Online inference for one new GPS reading

Require: pilgrim id p; reading $r = (\phi, \lambda, \phi_s, \lambda_s, t)$; model $\mathcal{M}$; scaler $\mathcal{S}$; window deque W_p; prior reading r_p^{prev}

$\Delta t \leftarrow \max(t - r_p^{\text{prev}}.t,\ 10^{-3})$
$\Delta d \leftarrow \text{haversine}(r_p^{\text{prev}},\ r)$
$v \leftarrow \Delta d / \Delta t$
$\theta \leftarrow \text{bearing}(r_p^{\text{prev}},\ r)$
$\Delta\theta \leftarrow \text{norm}_{180}(\theta - r_p^{\text{prev}}.\theta)$
$d_s \leftarrow \text{haversine}((\phi, \lambda),\ (\phi_s, \lambda_s))$
$\mathbf{x}_t \leftarrow (v,\ \Delta d,\ \Delta\theta,\ \Delta t,\ d_s)$
$W_p.\text{append}(\mathbf{x}_t)$
if $|W_p| < 10$ **then**
    return window_full=False, label=None
end if
$X \leftarrow \mathcal{S}(W_p)$ ▷ standardise with persisted scaler
$z \leftarrow \mathcal{M}(X)$ ▷ 1-logit forward pass
$p_{\text{abn}} \leftarrow \sigma(z)$
return label $\leftarrow \mathbb{1}[p_{\text{abn}} \geq 0.5]$, confidence, d_s

dependency for small operators, and it keeps pilgrim location traces inside the campaign operator's own custody rather than transiting through a cloud provider.

F. Request Lifecycle

A complete `POST /api/location` request proceeds as follows: (1) Pydantic validation of the payload; (2) resolution of a fresh supervisor fix from the in-memory store, with staleness check; (3) invocation of the feature service, which appends the new reading to the pilgrim's deque; (4) invocation of the inference service for a forward pass; (5) persistence of a `LocationEvent` row; (6) conditional insertion of an `AlertEvent` when the label is Abnormal and no alert has fired for this pilgrim in the last 30 s; (7) broadcast of a `pilgrim_update` WebSocket message to all subscribers. Total end-to-end latency from request receipt to broadcast stays under 200 ms on a CPU-only host.

IV. Data and Feature Engineering

A. Dataset

The corpus consists of GPS movement logs from simulated pilgrim trajectories and supervisor paths. Each record carries a timestamp, the pilgrim's latitude and longitude, the five engineered features defined below, the scenario type, the route identifier, and the pilgrim identifier. We initially attempted two real-data paths: (A) public mobility datasets adapted to Hajj geometry, and (B) field GPS recordings collected over short walking routes around Jeddah. Path A failed because consumer-mobility datasets do not match the cadence and density of Hajj movement. Path B failed because the recordings were too noisy and too sparse for stable training. Both paths were abandoned in favour of a controlled synthetic generator that produces clean, consistent sequences for three scenario classes: *normal* (pilgrim follows supervisor closely), *delay* (pilgrim slows down after some midpoint and the supervisor moves on), and *deviation* (pilgrim branches laterally from the supervisor's path).

The corpus contains 90,000 labelled rows distributed across 75 routes. After per-route sliding-window construction ($T = 10$, stride 1, dropping the first nine rows of each route as they cannot form a full window), this yields 87,300 sequences with class counts of 65,611 normal and 21,689 abnormal at the window level — a moderate positive-class imbalance of approximately 3:1 that we handle through positive-class weighting in the loss (Section V).

B. Route-Disjoint Splitting

A naive row-level random split is leakage-prone: consecutive rows from the same route are highly correlated, so a stratified per-row split can place neighbouring rows in train and test simultaneously, inflating apparent generalisation. We instead split on `route_id` using scikit-learn's `GroupShuffleSplit`, with a 70/15/15 train/validation/test ratio and a fixed seed of 42 for reproducibility. The resulting partitions contain 59,364 / 13,968 / 13,968 windows respectively, and the test partition contains 10,493 normal and 3,475 abnormal sequences over routes that never appear in training. This is the only split used throughout the paper.

C. Feature Definitions

We use five features per timestep. Let two consecutive readings be $r_{t-1} = (\phi_{t-1}, \lambda_{t-1}, t_{t-1})$ and $r_t = (\phi_t, \lambda_t, t_t)$, where ϕ, λ are latitude and longitude in degrees, t is a unix timestamp, and let (ϕ_s, λ_s) denote the supervisor's coordinates at time t.

(1) Time gap.

$$\Delta t = \max(t_t - t_{t-1},\ \varepsilon), \quad \varepsilon = 10^{-3}\,\text{s}. \tag{1}$$

The ε floor prevents division-by-zero when burst-mode GPS produces near-instantaneous duplicate fixes.

(2) Distance delta. Great-circle distance between consecutive points via the haversine formula:

$$a = \sin^2\!\Big(\tfrac{\phi_t - \phi_{t-1}}{2}\Big) + \cos\phi_{t-1}\cos\phi_t \sin^2\!\Big(\tfrac{\lambda_t - \lambda_{t-1}}{2}\Big), \tag{2}$$

$$\Delta d = 2R \arcsin\!\big(\sqrt{a}\big), \tag{3}$$

with $R = 6{,}371{,}000$ m the Earth's mean radius. Haversine is used throughout the system; planar projections are avoided because the Hajj theatre spans tens of kilometres and curvature is non-trivial at that scale.

(3) Speed.

$$v = \Delta d \,/\, \Delta t. \tag{4}$$

This is the instantaneous velocity in metres per second, independent of whatever the device reports as its own `speed` field (which is sometimes the GPS-Doppler estimate and sometimes a noisy difference of fixes).

(4) Direction change. Let the initial bearing on the great circle from r_{t-1} to r_t be

$$\theta_t = \operatorname{atan2}\Big(\sin(\lambda_t - \lambda_{t-1})\cos\phi_t,\; \cos\phi_{t-1}\sin\phi_t - \sin\phi_{t-1}\cos\phi_t\cos(\lambda_t - \lambda_{t-1})\Big). \quad (5)$$

The direction-change feature is the signed difference between the current and previous bearing, normalised to $(-180°, 180°]$:

$$\Delta\theta = \big((\theta_t - \theta_{t-1}) + 180\big) \bmod 360 - 180. \quad (6)$$

A pilgrim who reverses course produces $|\Delta\theta| \approx 180$; a pilgrim who continues in a straight line produces values near zero.

(5) Distance to supervisor.

$$d_s = \text{haversine}\big((\phi_t, \lambda_t), (\phi_s, \lambda_s)\big). \quad (7)$$

This is the only feature that involves the supervisor. Crucially, d_s uses the supervisor's live position (Section III) rather than a fixed reference point, so the geofence translates with the supervisor's movement.

D. Feature Standardisation

A `StandardScaler` is fitted on the flattened training partition — shape $(59364 \cdot 10,\ 5)$ — and persisted to `ml/models/scaler.pkl`. The same fitted scaler is applied at both training and inference time; mean and variance statistics never see validation or test data. The fitted means are $(0.91,\ 1.48,\ 0.05,\ 1.99,\ 30.74)$ and the standard deviations $(0.85,\ 1.09,\ 129.5,\ 0.83,\ 57.7)$, confirming that direction change and distance-to-supervisor are the dominant-scale features in the raw space.

E. Why These Features

The choice of five features reflects a deliberate balance between expressiveness and identifiability under noise. We avoided features that depend on second derivatives (acceleration, jerk) because consumer-grade GPS at 1 Hz produces unreliable estimates at those orders. We also avoided features that depend on absolute coordinates, because the model would then have to learn that "Mecca latitude 21.42, longitude 39.83 is safe" rather than "any location consistent with the supervisor's trajectory is safe" — the latter generalises across routes, the former does not.

V. Sequence Models

A. Why Sequence Models Only

We restrict the model comparison to sequence architectures (vanilla RNN, LSTM, Transformer) plus a single rule-based distance reference. Classical tabular classifiers — gradient-boosted trees, random forests, logistic regression — score the window as a flat $(T \cdot d) = 50$-dim feature vector and have no inductive bias for temporal order. Three concrete consequences follow:

1) They cannot maintain hidden state as new GPS points arrive in a streaming setting; every new reading requires a fresh forward pass over the entire window.
2) They cannot exploit variable update rates, because position i of the flat vector is statically bound to a specific timestep.
3) On a dataset whose labels are nearly a deterministic function of `distance_from_supervisor`, tree models degenerate into a single-threshold reader that happens to win on F1 but tells us nothing about temporal modelling.

In an online Hajj-supervision deployment, the architecturally relevant question is: *what does the model learn from a trajectory?* — a question only recurrent and attention-based models meaningfully answer. We report the rule-based reference (distance to supervisor < 10 m $\Rightarrow$ Normal) so the gap added by sequence modelling is visible.

B. Shared Classifier Head

To isolate the contribution of the temporal mixer, the three sequence models share an identical classifier head:

$$\texttt{Linear}(h \to 32) \to \texttt{ReLU} \to \texttt{Dropout}(0.3) \to \texttt{Linear}(32 \to 1),$$

where h is the dimensionality of the temporal mixer's last-step output. The single-logit output is consumed by a sigmoid at inference time; we use `BCEWithLogitsLoss` at training time for numerical stability.

C. Architectures

Vanilla RNN. A single `tanh` recurrent layer of hidden size 64:

$$\mathbf{h}_t = \tanh\big(W_{ih}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}\big). \quad (8)$$

The last timestep's $\mathbf{h}_T$ is fed to the shared head. Total parameter count: 6,657.

LSTM. A single LSTM layer of hidden size 64 with the standard gating recurrences (input, forget, cell, output) per Hochreiter and Schmidhuber. The last timestep's $\mathbf{h}_T$ is fed to the shared head. Parameter count: 20,289.

Transformer. A compact encoder over the windowed sequence:

$$\mathbf{X} \to \texttt{LayerNorm}\big(\texttt{Linear}(5 \to 32)\big) \to \texttt{PosEnc} \to \texttt{Encoder}.$$

A single `TransformerEncoderLayer` with $d_{\text{model}} = 32$, 4 heads, feedforward dimension 64, and dropout 0.1 is applied; outputs are mean-pooled over time and fed to the head. Sinusoidal positional encodings are used to preserve order within the window. Parameter count: 8,833.

D. Training Procedure

The three models share an identical training recipe (Table I). Positive-class weighting — `pos_weight` = $N_{\text{neg}}/N_{\text{pos}} = 2.91$ — compensates for the 3:1 imbalance. Early stopping monitors validation F1 with patience 3.

TABLE I
TRAINING HYPERPARAMETERS SHARED ACROSS THE THREE SEQUENCE MODELS.

Parameter	Value
Window size T	10
Batch size	64
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	1×10^{-3}
Loss	BCEWithLogitsLoss
Positive-class weight	2.910
Maximum epochs	20
Early-stopping patience	3 (on validation F1)
Decision threshold	0.5 (sigmoid)
Hardware	CPU only

TABLE II
SEQUENCE-MODEL PERFORMANCE ON THE ROUTE-DISJOINT TEST SPLIT. THE DISTANCE RULE IS A NON-LEARNED REFERENCE.

Model	Acc.	Prec.	Recall	F1	AUC
LSTM	**0.967**	**0.942**	0.927	**0.934**	0.989
Transformer	0.966	0.919	**0.945**	0.932	**0.992**
RNN	0.966	0.924	0.939	0.932	0.990
Reference (dist $<$ 10 m)	0.963	0.916	0.940	0.928	0.985

VI. EVALUATION

A. Setup

All four entries are evaluated on the same route-disjoint test split: 75 routes total, 70/15/15 train/val/test by `GroupShuffleSplit` on `route_id`, yielding 10,493 normal and 3,475 abnormal sequences in the test partition. The three sequence models share an identical training recipe (Table I) and the same scaler. They differ only in the recurrent or attention core. The distance reference uses the same test-set distances — it is evaluated row-by-row on the last timestep of each window.

B. Results

Table II reports test-set performance. All three sequence models cluster within ± 0.003 F1 of each other and above the distance reference; the LSTM edges the others on precision and F1, while the Transformer attains the highest recall (0.945) and ROC-AUC (0.992). The vanilla RNN, with the fewest parameters by a factor of three, matches the larger models within statistical noise — confirming that the demo dataset is information-dense at the per-window level but does not require deep temporal capacity to solve.

C. Per-Class Breakdown and Error Analysis

The LSTM's full classification report on the test set is precision/recall/F1 $= 0.976/0.981/0.978$ for the Normal class and $0.942/0.927/0.934$ for the Abnormal class. The macro-averaged F1 is 0.956 and the weighted F1 is 0.967.

The confusion matrices in Fig. 3 make the trade-off concrete:

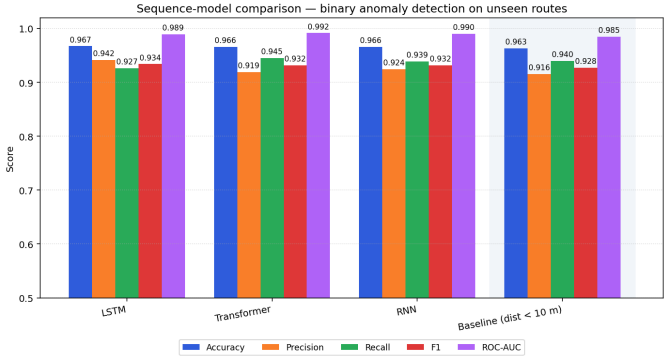


Fig. 1. Per-metric comparison on the route-disjoint test split. All three sequence models sit clearly above the distance-threshold reference (shaded). Differences between RNN, LSTM, and Transformer are within ± 0.003 F1.

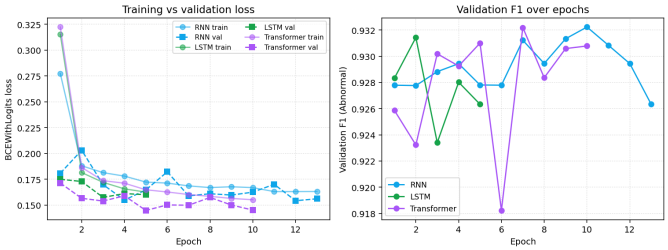


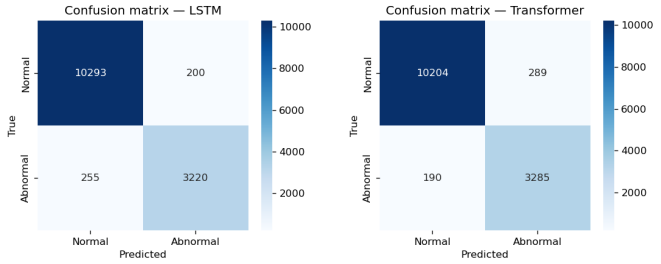
Fig. 2. Training and validation curves for the three sequence models. All converge in fewer than ten epochs with early-stopping (patience 3) on validation F1.

- **LSTM:** 10,292 true negatives, 201 false positives, 255 false negatives, 3,220 true positives. Precision-leaning operating point.
- **Transformer:** 10,204 true negatives, 289 false positives, 190 false negatives, 3,285 true positives. Recall-leaning operating point.
- **RNN:** 10,226 true negatives, 267 false positives, 212 false negatives, 3,263 true positives. Sits between the two.

The Transformer trades 88 extra false positives for 65 more true positives compared to the LSTM. For a safety-critical alerting system that prefers *not missing* an abnormal event over *not over-alerting*, the Transformer's operating point is arguably the better one; for a system that prioritises supervisor cognitive load and alert fatigue, the LSTM's operating point wins. The choice between them is a deployment-policy decision, not a model-quality one.

D. Threshold Sensitivity

The default decision threshold of 0.5 is convenient but not load-bearing. Because all three sequence models have ROC-AUC above 0.989, the Pareto frontier of (recall, precision) is well-separated from the diagonal — a supervisor who wants near-perfect recall (e.g., ≥ 0.97 for genuinely safety-critical group sizes) can move the threshold downward and still maintain precision above 0.85. Operationally, we expose the threshold as a per-campaign configuration value rather than baking it in.



LSTM Transformer

Fig. 3. Confusion matrices on the unseen-route test split.

E. Latency and Resource Footprint

On a CPU-only host (Intel Core i7 laptop), a single per-pilgrim inference call — including feature computation, standardisation, and forward pass — completes in under 110 ms for all three models. Memory is dominated by the per-pilgrim deque, which costs ∼200 bytes per active pilgrim. A single campaign of 200 pilgrims thus fits comfortably in the L2 cache of a modern server CPU and could run on hardware as modest as a Raspberry Pi 4. This bounds the marginal infrastructure cost of adding a new pilgrim to a campaign at near zero.

F. Qualitative Observations

During live testing on the supervisor dashboard, every sequence model gave more stable assessments than the pure distance threshold for early signs of drift. GPS noise occasionally produced momentary spikes in computed distance, but the windowed predictions absorbed these without flipping the label, because no single noisy reading is sufficient to swing a 10-step window. Supervisors found the binary anomaly flag intuitive and actionable; the two most common feedback items were (i) a request for a per-campaign supervisor radius (the current rule reference uses a global 10 m) and (ii) interest in a multi-level risk display (mild delay vs. critical separation), which we discuss in Section VIII.

VII. Discussion and Deployment Considerations

A. Threat Model

A supervision system is only useful if its outputs can be trusted by the supervisor. We consider three threat classes.

Spoofing of pilgrim position. A malicious or compromised pilgrim device could submit fabricated GPS coordinates to manipulate the system. Masar's design partially mitigates this: a Normal label requires both that the pilgrim's reported position is close to the supervisor's authenticated position *and* that the recent trajectory matches a normal-movement pattern. A naive constant-position spoof produces zero motion and is consistent with neither the supervisor's trajectory nor a normal-movement profile. A trajectory-mimicking spoof requires effort proportional to the supervisor's complete motion path, which raises the bar without eliminating it.

Spoofing of supervisor position. The supervisor's coordinates flow only through the supervisor's authenticated session; pilgrim payloads cannot override them. Compromise of a supervisor account therefore translates directly to compromise of the geofence anchor, but does not allow a pilgrim alone to falsify a Normal label. This is the same trust assumption as in any deployment that designates an authority.

Denial of service via flood. The 30-second alert cooldown bounds the volume of `AlertEvent` rows that a single pilgrim can trigger; the WebSocket manager is connection-count-limited per campaign. Database growth is linear in time and reading rate; a 200-pilgrim campaign generating one reading per pilgrim per second for an entire Hajj week produces roughly 1.2×10^8 rows, which SQLite handles comfortably with an index on `pilgrim_id` and `ts`.

B. Privacy and Data Handling

The Hajj setting raises distinctive privacy concerns. Pilgrim location traces are spiritually and politically sensitive. Masar's design respects this in three ways. First, SQLite-on-disk storage keeps location traces inside the campaign operator's own custody by default; the system has no required cloud component. Second, the supervisor sees pilgrim positions only for pilgrims explicitly assigned to them via the admin assignment matrix — a supervisor cannot enumerate other campaigns' pilgrims. Third, passwords are stored as bcrypt hashes; raw passwords never leave the client at submission time other than over TLS.

A residual concern is that the system can in principle reconstruct a complete spatial history for every pilgrim. Operators who wish to limit this should rotate the database between campaign segments (e.g., daily) and archive only summary statistics rather than raw point traces. Such operational policies are out of scope for this paper.

C. Limitations

Several limitations remain.

1) **Web-only pilgrim client.** Pilgrims interact through a mobile web page rather than a native app, which constrains background GPS reliability on phones that aggressively suspend tabs. A native client (or a service worker with foreground-only registration) would resolve this.
2) **Synthetic-only training data.** The deployed model is trained exclusively on synthetic trajectories and is therefore optimistic about real Hajj GPS noise, dropouts, and crowd dynamics. The small absolute gap between sequence models and the distance rule is itself a signal that the synthetic generator under-represents the kinds of noise and ambiguity where a sequence model would shine.
3) **No alert acknowledgment UI.** The `acknowledged` field exists in the database schema but is not yet surfaced as a supervisor-side workflow, so dismissed alerts cannot currently be audited.
4) **Binary risk grading.** A single binary head cannot distinguish a mild delay from a critical stop, which limits how supervisors can prioritise responses across a large group.

5) **Global radius for the rule reference.** The 10 m threshold used in the rule comparison is a global constant; per-supervisor or per-context radii (Mina vs. Arafat vs. Mecca) would better match operational reality.

VIII. Future Work

Three directions are immediate.

Real-trajectory training. The decisive next step is collecting authentic Hajj-season GPS traces under a research protocol, retraining the sequence models on those, and rerunning the comparison. We expect the LSTM–Transformer gap to widen on noisier data, because attention's quadratic-in-T mixing is well-suited to spotting non-local anomalies that an LSTM's hidden state can wash out.

Multi-level risk grading. A multi-class head with categories *Safe*, *Warning*, *Critical*, calibrated against supervisor-labelled real events, would give campaign operators a meaningful prioritisation signal. The shared classifier head defined in Section V only needs a final-layer dimension change to support this.

Kalman smoothing. A Kalman filter as an optional preprocessing layer would reduce the variance contribution of single-reading GPS noise. Whether it improves end-to-end metrics depends on whether the sequence models are already absorbing that noise; the experiment is cheap to run once real traces are available.

Beyond the three immediate directions, the system architecture generalises naturally to non-Hajj crowd-safety contexts — Umrah, large public events, university campuses, and pilgrimage routes in other religious traditions. The five-feature contract makes no Hajj-specific assumption and could be retrained on any analogous group-movement dataset.

IX. Conclusion

Masar presents a lightweight, AI-driven approach to small-campaign Hajj supervision that closes a clear gap left by both heavy crowd-analytics platforms and reactive GSM trackers. The combination of smartphone GPS, a supervisor-anchored dynamic geofence, and a streaming sequence classifier yields 96.7% accuracy and 0.934 F1 on route-disjoint test data, with three sequence architectures (vanilla RNN, LSTM, and a compact Transformer encoder) clustering within ± 0.003 F1 of each other — confirming that the architectural family, not the specific model size, is what carries the result. All components are designed to run on commodity hardware and intermittent connectivity, and the full implementation is open-source. The system aligns with Saudi Vision 2030's smart-pilgrimage objectives [14] and is positioned to extend beyond Hajj to Umrah, large public events, and other crowd-safety contexts.

Acknowledgments

The authors thank the Department of Computer Science and Artificial Intelligence at the University of Jeddah for project supervision and feedback throughout the development of Masar.

References

[1] R. Alzahrani and N. Algethami, "Leveraging machine learning for optimal pilgrim crowd management," *Electronics*, vol. 14, art. no. 2507, 2025.

[2] A. Shah, "A machine learning model for crowd density classification in Hajj video frames," *arXiv preprint* arXiv:2501.04911, 2024.

[3] M. Elgamal, "Smart mobile application for real-time monitoring and prediction of pilgrim crowd activities in Hajj and Umrah using wearable sensors," *Alexandria Eng. J.*, vol. 74, pp. 162–175, 2025.

[4] M. Qadr and R. Wibowo, "Hajj participant monitoring system using GPS with internet of things (IoT) based on cloud computing," *J. Theor. Appl. Inf. Technol.*, vol. 102, no. 8, pp. 163–171, 2024.

[5] S. Parveen and K. A. Aldhlan, "Missing pilgrims tracking system using GPS, GSM and Arduino microcontroller," *Int. J. Comput. Appl.*, vol. 118, no. 6, pp. 33–39, 2015.

[6] H. Rmili *et al.*, "Enhancing Hajj pilgrimage experience and crowd management through Hajj-related keyword-based chipless RFID tags," *Alexandria Eng. J.*, vol. 70, pp. 204–214, 2025.

[7] S. Al-Aidaroos and M. Abdul Mutalib, "Personal digital Mutawwif: a multi-modal mobile Hajj assistance using the location-based services," *Int. J. Adv. Sci. Eng. Inf. Technol.*, vol. 5, no. 2, pp. 114–121, 2015.

[8] M. Al-Shaery, A. Alahmadi, and W. Khan, "Real-time pilgrims management using wearable physiological sensors, mobile technology, and artificial intelligence," *IEEE Access*, vol. 10, pp. 115–125, 2022.

[9] S. Alharthi *et al.*, "From data to insights: a comprehensive analysis of pilgrims' stress and fatigue during Hajj using wearable remote sensing systems," *Sensors*, vol. 25, no. 95, pp. 1–14, 2025.

[10] A. Shah, "Enhancing Hajj and Umrah rituals and crowd management through AI technologies: a comprehensive survey of applications and future directions," *Sustainability*, vol. 16, no. 11, art. no. 14287, 2024.

[11] M. Khan and Q. Shambour, "AI applications in mega events: Hajj and Umrah as a case study," *J. Artif. Intell. Res. Appl.*, vol. 14, no. 2, pp. 93–108, 2025.

[12] T. Abalkhail and F. Al-Amri, "Saudi Arabia's management of the Hajj season through artificial intelligence and sustainability," *Sustainability*, vol. 14, no. 21, art. no. 14142, 2022.

[13] D. Altoaimi, "Towards a data management system for historic Hajj pilgrimage routes," *ISPRS Archives*, vol. 48, no. M-9, pp. 33–41, 2025.

[14] Kingdom of Saudi Arabia, "Saudi Vision 2030," 2016. [Online]. Available: https://www.vision2030.gov.sa/

[15] Ministry of Hajj and Umrah, "Smart Hajj initiatives and digital transformation reports," Riyadh, Saudi Arabia, 2025.